

Лабораторная работа № 1

Тема: Создание первого серверного приложения на Express.js

Цель: Научиться создавать Node.js-проект, устанавливать Express.js, запускать HTTP-сервер, создавать GET-маршруты и возвращать клиенту текстовые и JSON-ответы.

Практическое задание

Создать серверное приложение **Internet Shop API**, которое в следующих лабораторных работах будет постепенно расширяться до backend учебного интернет-магазина.

На данном этапе база данных не используется. Информация о товарах хранится непосредственно в JavaScript-коде.

Последовательность выполнения:

1. Создайте новый каталог проекта.

```
mkdir express-shop  
cd express-shop
```

Откройте его в Visual Studio Code:

```
code .
```

2. Инициализируйте Node.js-проект.

```
npm init -y
```

Откройте созданный package.json и изучите его содержимое.

Добавьте использование ES Modules:

```
{  
  "type": "module"  
}
```

Поле необходимо добавить в существующий объект package.json, а не заменять им всё содержимое файла.

3. Установите Express.js.

```
npm install express
```

После установки убедитесь, что:

- появился каталог node_modules;
- появился package-lock.json;
- в package.json появился раздел dependencies;
- среди зависимостей присутствует Express.

4. Создайте файл app.js.

Подключите Express:

```
import express from "express";
```

Создайте объект приложения:

```
const app = express();
```

Создайте константу:

```
const PORT = 3000;
```

5. Реализуйте запуск HTTP-сервера.

Сервер должен запускаться на порту 3000.

После успешного запуска в терминале должно выводиться сообщение:

Server: http://localhost:3000

Запустите приложение:

node app.js

Откройте указанный адрес в браузере.

6. Создайте главную страницу API.

Для запроса:

GET /

сервер должен вернуть:

"Internet Shop API"

Для ответа используйте `res.send()`.

7. Создайте маршрут с описанием проекта.

Для запроса:

GET /about

верните небольшой текст с описанием вашего учебного интернет-магазина.

Например:

"Учебный REST API интернет-магазина компьютерной техники"

8. Создайте маршрут состояния сервера.

Для запроса:

GET /api/status

сервер должен вернуть JSON:

```
{
  "status": "ok",
  "message": "Server is running"
}
```

Для формирования ответа используйте `res.json()`.

9. Создайте информацию о магазине.

Создайте маршрут:

GET /api/shop

Он должен возвращать объект примерно такой структуры:

```
{
  "name": "Tech Store",
  "description": "Учебный интернет-магазин",
  "currency": "KZT"
}
```

Название и описание магазина студент выбирает самостоятельно.

10. Создайте массив товаров.

В app.js создайте массив минимум из пяти объектов.

Каждый товар должен иметь поля:

- id;
- name;
- price;
- category;
- inStock.

Например:

```
const products = [  
  {  
    id: 1,  
    name: "Keyboard",  
    price: 15000,  
    category: "Accessories",  
    inStock: true  
  },  
  {  
    id: 2,  
    name: "Mouse",  
    price: 8000,  
    category: "Accessories",  
    inStock: true  
  }  
];
```

Остальные товары придумайте самостоятельно.

11. Создайте маршрут каталога.

Для запроса:

GET /api/products

сервер должен возвращать весь массив products в формате JSON.

12. Создайте маршрут категорий.

Создайте массив минимум из трёх категорий.

Например:

```
const categories = [  
  "Computers",  
  "Accessories",  
  "Monitors"  
];
```

Для запроса:

GET /api/categories

верните этот массив в формате JSON.

13. Добавьте прт-команду запуска.

В раздел scripts файла package.json добавьте:

```
{  
  "scripts": {
```

```
    "start": "node app.js"
  }
}
```

После этого сервер должен запускаться командой:
npm start

14. Проверьте работу приложения.

Последовательно выполните следующие запросы:

- GET /
- GET /about
- GET /api/status
- GET /api/shop
- GET /api/products
- GET /api/categories

Проверьте:

- отвечает ли сервер;
- соответствует ли ответ нужному маршруту;
- являются ли ответы /api/... корректным JSON;
- содержится ли в каталоге минимум пять товаров;
- возвращаются ли все категории.

Требования к результату

Минимальная структура проекта:

```
express-shop/
├── node_modules/
├── app.js
├── package.json
└── package-lock.json
```

Приложение должно:

- использовать Express.js;
- использовать синтаксис ES Modules;
- запускаться командой npm start;
- работать на порту 3000;
- содержать шесть обязательных GET-маршрутов;
- возвращать текстовые ответы через res.send();
- возвращать объекты и массивы через res.json();
- содержать минимум пять товаров;
- содержать минимум три категории;
- запускаться без ошибок.

PS: Что бы сервер перезапускался автоматически при изменении кода, нужно внести изменение в package.json:

```
"scripts": {
  "start": "node app.js",
```

```
"dev": "node --watch app.js"
```

```
}
```

Теперь вместо:

```
node app.js
```

запускать так:

```
npm run dev
```